

Strutture dati

ArrayList

Come usare liste dinamiche quando il numero di elementi può cambiare.

Perché usare ArrayList

Un array ha una dimensione fissa. Se lo crei con 3 elementi, resta di 3 elementi.

`ArrayList` è una lista dinamica: puoi aggiungere e rimuovere elementi mentre il programma gira.

Prima devi importarla:

Creare una ArrayList

```
public class ListaNomi {  
    public static void main(String[] args) {  
        ArrayList nomi = new ArrayList<>();  
  
        nomi.add("Luca");  
        nomi.add("Sara");  
        nomi.add("Mina");  
  
        System.out.println(nomi);  
    }  
}
```

Output:

```
[Luca, Sara, Mina]
```

`ArrayList` significa: "lista di stringhe".

Aggiungere elementi

Usa `add` :

```
ArrayList prodotti = new ArrayList<>();  
  
prodotti.add("Pane");  
prodotti.add("Latte");  
prodotti.add("Mele");
```

Ogni `add` aggiunge un elemento in fondo alla lista.

Leggere elementi

Usa `get` con l'indice.

```
ArrayList nomi = new ArrayList<>();  
nomi.add("Luca");  
nomi.add("Sara");  
  
System.out.println(nomi.get(0));
```

Output:

```
Luca
```

Anche qui gli indici partono da zero.

Modificare elementi

Usa `set` .

```
ArrayList nomi = new ArrayList<>();  
nomi.add("Luca");  
nomi.add("Sara");  
  
nomi.set(1, "Giulia");  
  
System.out.println(nomi);
```

Output:

```
[Luca, Giulia]
```

Rimuovere elementi

Usa `remove` .

```
ArrayList prodotti = new ArrayList<>();  
prodotti.add("Pane");  
prodotti.add("Latte");  
prodotti.add("Mele");  
  
prodotti.remove("Latte");  
  
System.out.println(prodotti);
```

Output:

```
[Pane, Mele]
```

Puoi rimuovere anche per indice:

```
prodotti.remove(0);
```

size

`size()` restituisce il numero di elementi.

```
ArrayList nomi = new ArrayList<>();  
nomi.add("Luca");  
nomi.add("Sara");  
  
System.out.println(nomi.size());
```

Output:

```
2
```

Negli array usi `length` . Nelle `ArrayList` usi `size()` .

Array o ArrayList?

Usa un array quando:

- la dimensione è fissa
- vuoi una struttura semplice
- stai facendo esercizi sugli indici

Usa `ArrayList` quando:

- il numero di elementi può cambiare
- devi aggiungere o rimuovere valori
- vuoi una lista più comoda da gestire

Esempio completo

```
public class Spesa {  
    public static void main(String[] args) {  
        ArrayList lista = new ArrayList<>();  
  
        lista.add("Pane");  
        lista.add("Latte");  
        lista.add("Mele");  
  
        lista.remove("Latte");  
        lista.add("Pasta");  
  
        for (int i = 0; i < lista.size(); i++) {  
            System.out.println("- " + lista.get(i));  
        }  
    }  
}
```

Output:

```
- Pane  
- Mele  
- Pasta
```

ArrayList è una delle strutture più usate in Java quando lavori con gruppi di dati.

Array

Come raccogliere più valori dello stesso tipo in una struttura ordinata.

Cosa è un array

Un array raccoglie più valori dello stesso tipo in una sequenza ordinata.

Pensalo come una fila di caselle numerate.

```
int[] voti = {7, 8, 6, 9};
```

Qui `voti` contiene quattro numeri.

Gli indici partono da zero

Per leggere un elemento, usi il suo indice.

```
int[] voti = {7, 8, 6, 9};  
  
System.out.println(voti[0]);  
System.out.println(voti[1]);
```

Output:

```
7  
8
```

Il primo elemento ha indice `0`, non `1`.

Modificare un elemento

Puoi cambiare il valore di una casella:

```
int[] voti = {7, 8, 6, 9};  
  
voti[2] = 10;  
  
System.out.println(voti[2]);
```

Output:

10

Prima `voti[2]` valeva `6` . Dopo l'assegnazione vale `10` .

Creare un array vuoto

Puoi creare un array indicando la sua dimensione:

```
String[] nomi = new String[3];  
  
nomi[0] = "Luca";  
nomi[1] = "Sara";  
nomi[2] = "Mina";
```

La dimensione di un array non cambia dopo la creazione.

Se crei un array di 3 elementi, avrà sempre 3 caselle.

length

`length` contiene il numero di elementi.

```
String[] nomi = {"Luca", "Sara", "Mina"};  
  
System.out.println(nomi.length);
```

Output:

3

Negli array `length` non ha parentesi. Non scrivere `length()` .

Attraversare un array con for

```
String[] nomi = {"Luca", "Sara", "Mina"};

for (int i = 0; i < nomi.length; i++) {
    System.out.println(nomi[i]);
}
```

Output:

```
Luca
Sara
Mina
```

Questa forma è utile quando ti serve l'indice.

Errore comune: indice fuori limite

```
int[] numeri = {10, 20, 30};
System.out.println(numeri[3]); // errore
```

Gli indici validi sono:

- 0
- 1
- 2

3 è fuori dall'array.

Java segnala un errore chiamato `ArrayIndexOutOfBoundsException`.

Esempio: media dei voti

```
public class Media {  
    public static void main(String[] args) {  
        int[] voti = {7, 8, 6, 9};  
        int somma = 0;  
  
        for (int i = 0; i < voti.length; i++) {  
            somma = somma + voti[i];  
        }  
  
        double media = (double) somma / voti.length;  
        System.out.println("Media: " + media);  
    }  
}
```

Output:

```
Media: 7.5
```

L'array è utile quando hai molti valori simili e vuoi trattarli insieme.

HashMap

Come associare chiavi e valori per rappresentare dati cercabili.

Cosa è una HashMap

Una **HashMap** collega una **chiave** a un **valore**.

Pensala come un piccolo dizionario:

- chiave: parola
- valore: significato

Oppure come una rubrica:

- chiave: nome
- valore: numero di telefono

Prima devi importarla:

Creare una HashMap

```
public class Rubrica {  
    public static void main(String[] args) {  
        HashMap telefoni = new HashMap<>();  
  
        telefoni.put("Luca", "3331234567");  
        telefoni.put("Sara", "3337654321");  
  
        System.out.println(telefoni);  
    }  
}
```

`HashMap` significa: chiavi di tipo `String` , valori di tipo `String` .

Aggiungere con put

Usa `put` per aggiungere una coppia chiave-valore.

```
HashMap eta = new HashMap<>();  
  
eta.put("Luca", 20);  
eta.put("Sara", 25);
```

Se usi una chiave già presente, il valore viene sostituito.

```
eta.put("Luca", 21);
```

Ora "Luca" vale 21 .

Leggere con get

Usa `get` per recuperare il valore associato a una chiave.

```
HashMap eta = new HashMap<>();  
eta.put("Luca", 20);  
  
System.out.println(eta.get("Luca"));
```

Output:

```
20
```

Se la chiave non esiste, `get` restituisce `null` .

Controllare se una chiave esiste

Prima di leggere, puoi usare `containsKey` .

```
HashMap eta = new HashMap<>();  
eta.put("Luca", 20);  
  
if (eta.containsKey("Mina")) {  
    System.out.println(eta.get("Mina"));  
} else {  
    System.out.println("Mina non e presente.");  
}
```

Output:

Mina non è presente.

Rimuovere una chiave

Usa `remove` .

```
HashMap eta = new HashMap<>();  
eta.put("Luca", 20);  
eta.put("Sara", 25);  
  
eta.remove("Luca");  
  
System.out.println(eta);
```

Output:

```
{Sara=25}
```

Attraversare le chiavi

Puoi leggere tutte le chiavi con `keySet()` .

```
HashMap eta = new HashMap<>();  
eta.put("Luca", 20);  
eta.put("Sara", 25);  
  
for (String nome : eta.keySet()) {  
    System.out.println(nome + ": " + eta.get(nome));  
}
```

Output possibile:

Luca: 20
Sara: 25

L'ordine non è garantito. Una `HashMap` serve a cercare per chiave, non a mantenere un ordine preciso.

Esempio: prezzi dei prodotti

```
public class Prezzi {  
    public static void main(String[] args) {  
        HashMap prezzi = new HashMap<>();  
  
        prezzi.put("pane", 1.50);  
        prezzi.put("latte", 1.20);  
        prezzi.put("pasta", 0.90);  
  
        String prodotto = "latte";  
  
        if (prezzi.containsKey(prodotto)) {  
            System.out.println("Prezzo: " + prezzi.get(prodotto));  
        } else {  
            System.out.println("Prodotto non trovato.");  
        }  
    }  
}
```

`HashMap` è utile quando vuoi cercare rapidamente un valore partendo da una chiave.

Set

Come conservare elementi unici senza duplicati.

Cosa è un Set

Un `Set` è una collezione che contiene elementi unici.

Se provi ad aggiungere due volte lo stesso valore, il duplicato non viene inserito.

Per iniziare useremo `HashSet`.

Creare un HashSet

```
public class NomiUnici {  
    public static void main(String[] args) {  
        HashSet nomi = new HashSet<>();  
  
        nomi.add("Luca");  
        nomi.add("Sara");  
        nomi.add("Luca");  
  
        System.out.println(nomi);  
    }  
}
```

Output possibile:

[Luca, Sara]

`"Luca"` compare una sola volta.

Aggiungere elementi

Usa `add`.

```
HashSet parole = new HashSet<>();

parole.add("java");
parole.add("codice");
parole.add("java");
```

Il secondo `"java"` viene ignorato perché esiste già.

Controllare se un elemento esiste

Usa `contains`.

```
HashSet invitati = new HashSet<>();
invitati.add("Luca");
invitati.add("Sara");

if (invitati.contains("Sara")) {
    System.out.println("Sara è nella lista.");
}
```

Output:

```
Sara è nella lista.
```

Rimuovere elementi

Usa `remove`.

```
HashSet invitati = new HashSet<>();
invitati.add("Luca");
invitati.add("Sara");

invitati.remove("Luca");

System.out.println(invitati);
```

Output:

```
[Sara]
```

Attraversare un Set

Puoi usare il ciclo for-each:

```
HashSet linguaggi = new HashSet<>();
linguaggi.add("Java");
linguaggi.add("Python");
linguaggi.add("PHP");

for (String linguaggio : linguaggi) {
    System.out.println(linguaggio);
}
```

L'ordine non è garantito. Un `HashSet` pensa all'unicità, non alla posizione.

Set, ArrayList o array?

Usa un array quando hai una dimensione fissa.

Usa `ArrayList` quando vuoi una lista ordinata che può crescere.

Usa `HashSet` quando ti interessa evitare duplicati e controllare rapidamente se un elemento esiste.

Esempio: parole uniche

```
public class ParoleUniche {  
    public static void main(String[] args) {  
        String[] parole = {"java", "codice", "java", "metodo", "codice"};  
        HashSet uniche = new HashSet<>();  
  
        for (int i = 0; i < parole.length; i++) {  
            uniche.add(parole[i]);  
        }  
  
        System.out.println(uniche);  
    }  
}
```

Output possibile:

```
[java, metodo, codice]
```

Il risultato contiene ogni parola una sola volta.